PROJECT REPORT

GitOps Workflow Using ArgoCD and Kubernetes

Prepared by: Vancha Rishika Reddy | **Repository:** github.com/chinnu9729/argocd-gitops-project

1. Introduction

GitOps is an approach to managing application deployments and infrastructure configuration by treating a Git repository as the source of truth. Instead of manually applying every configuration change to a cluster, a GitOps tool monitors the repository and reconciles the cluster to match the declared configuration. This project implements a GitOps workflow using ArgoCD and a lightweight K3s Kubernetes cluster hosted on an AWS EC2 Ubuntu 24.04 instance. A sample Nginx application is deployed and managed through Kubernetes manifests stored in GitHub.

2. Abstract

The objective of this project is to demonstrate automated Kubernetes application delivery using GitOps principles. Kubernetes Deployment and Service manifests are maintained in a public GitHub repository and connected to an ArgoCD application. ArgoCD monitors the main branch and the `manifests/` directory, then automatically synchronizes changes to the cluster. The project demonstrates scaling the Nginx Deployment from two replicas to three by changing the replica count in Git. ArgoCD detects the commit and Kubernetes creates the additional Pod. The final state is verified as three ready, running Pods.

3. Tools Used

Tool / Technology	Purpose
AWS EC2	Hosts the Ubuntu-based Kubernetes environment.
Ubuntu 24.04	Operating system for the project server.
K3s / Kubernetes	Runs and manages the containerized Nginx application.
ArgoCD	Monitors Git and automatically synchronizes Kubernetes resources.
Git / GitHub	Version-controls and stores the Kubernetes manifests and project documentation.
Docker / Nginx	Container technology and sample web application used for deployment.
kubectl	Command-line tool used to inspect and verify cluster resources.

4. Steps Involved in Building the Project

- **Infrastructure setup:** Created an Ubuntu 24.04 EC2 instance on AWS and installed K3s to provide a lightweight Kubernetes cluster. Configured kubectl to communicate with the cluster.
- **ArgoCD installation:** Installed ArgoCD in the `argocd` namespace and configured access to its dashboard through the EC2 instance.
- **Kubernetes manifests:** Created a Deployment manifest for the Nginx image `nginx:1.27-alpine` and a ClusterIP Service. The resources were placed in the `gitops-demo` namespace.
- **GitHub integration:** Pushed the manifests to `chinnu9729/argocd-gitops-project`. Created the ArgoCD application to monitor the `main` branch and `manifests/` path.
- **Automated synchronization:** Enabled automated sync, self-healing, and pruning so ArgoCD could reconcile the cluster with the configuration stored in Git.
- **Scaling demonstration:** Changed the Deployment replica count from 2 to 3 and committed the change. ArgoCD detected the new commit and synchronized the desired state, resulting in three Nginx Pods.
- **Verification:** Checked the ArgoCD application status and used kubectl commands such as `kubectl get deployments -n gitops-demo` and `kubectl get pods -n gitops-demo`. Confirmed the Deployment was 3/3 ready and all three Pods were Running. The cluster state was checked again after an EC2 restart.

5. Conclusion

The project successfully demonstrates a GitOps-based deployment workflow using ArgoCD, Kubernetes, AWS EC2, and GitHub. Kubernetes configuration was maintained in Git, and ArgoCD automatically synchronized repository changes with the cluster. The scaling test confirmed that changing the desired replica count in Git resulted in the expected three running Nginx Pods. This project provided practical experience with cloud infrastructure, container orchestration, version control, and continuous delivery using GitOps.